What survives when a paper leaves the lab: re-testing 22 robot-learning papers on a €1,350 rig, with an audit of our own pre-registration

Ravindu Somawansa
Shisso Robotics
ravindu.somawansa@gmail.com*

Abstract

Robot-learning papers are mostly shown in simulation, on costly arms or on data-centre GPUs. Does the claim survive on hardware a student can buy? We re-tested the central claim of 22 papers on a low-cost arm (SO-101) and an edge GPU (Jetson Orin NX), with the cheapest baseline run first and a pass mark fixed before each run. Of the 37 claims taken from the papers, 12 reproduce and 18 fail; the rest are not proven or not tested, and 8 further rows compare against our own baselines. Three classes cover the failures we saw: methods that miss a real-time budget on the edge GPU, zero-shot judges beaten by a trivial signal, and latent planners beaten by the commanded joint positions. We also audited ourselves. Every published number was rebuilt from its stored rows, the early gates were recomputed with one statistic, and commit order says which bars provably preceded their results — most, not all. The class assignment itself waits for a blind second reader. Both limits are in the paper, with their counts. Total compute: $45.10.

1 The register

A claim shown on a Franka-class arm, in simulation or on a data-centre GPU does not say whether it holds on the hardware most students and small labs own. The register is one entry per paper: the claim as the paper states it, the pass mark we fixed for our rig, our result and a verdict, with the numbers recomputable from published rows. It rests on three conditions. **Accessible hardware:** an SO-101 arm and a Jetson Orin NX 16 GB, €1,350 for the parts with a stated price (2 smaller parts not yet priced). **Cheapest baseline first:** before a learned model is credited, the trivial signal it must beat (copy-last, the commanded joints, episode length) is run on the same rows. **Bar before the run:** the pass mark is committed before the result; §2 audits how far that holds. The papers were picked for relevance to the stack this rig runs — latent world models, VLA policies, failure monitors and judges, action chunking, edge latency — and entered in the register before testing. The register file lists the papers scheduled next. They are not a sample of the field (§5). Of the 22 papers, 9 were shown in simulation, 7 on rigs above $20k, 3 at $5–20k, 2 below $5k and 1 on an unstated setup. Three things come out of this. First, the register: 45 claim rows over 22 papers, each with the claim as the paper states it, a bar fixed for this rig, a verdict, and the rows it is recomputed from. Second, a commit-order audit of the study's own pre-registration, with the provable and the unprovable counts side by side. Third, three proposed failure classes, with a blind two-reader protocol committed before any labelling. Everything, the audit's own evidence included, sits in one repository.

2 Method, and an audit of our own pre-registration

Each gate of the experiment log has a bar and a result. We find, in git history, the first commit containing each, and classify the gate by commit order alone (`git merge-base --is-ancestor`) — never from a note, never from a timestamp. Of the 89 gate rows of the log as of 2026-10-31, **58 are bar-first, 6 same-commit, 0 result-first, 23 no-bar and 2 not-run** (a bar, never a result). The 93 rows of the log today give 61 / 7 / 0 / 23 / 2. "Same-commit" means the bar and the result first appear in the same commit of this repository; it is not evidence that the result came first, nor that it did not. A first pass by text search alone had read 36 bar-first, 31 same-commit, 19 no-bar and 3 unresolved. Tracing each same-commit gate to its own protocol, plan, scorer or specification moved 21 to bar-first and 4 to no-bar (the bar cell itself declares none), and left 6. Those upgrades rest on a bar text found in an earlier commit of a file tied to the gate. The evidence line for each is in the audit table, so a reader can check it; the earlier bar-first rows were not re-audited for a result elsewhere in the history. One gate added on 2026-

*Every number with its source file: `github.com/ravediamond/paper-register-reproducibility`.

11-01 (the reward-model progress test) is same-commit by text search, but its protocol (`6241570`) predates the log commit; its result commit was not audited, so it stays same-commit and is an open candidate. **One statistic.** The early gates mixed point estimates and other intervals, so we recomputed 10 of them on the episode bootstrap (2,000 resamples, percentile 95 % CI) from stored rows. 3 verdicts flip somewhere inside their interval: the oscillation rate among failed episodes (24.0 %, CI 8.0–40.0 %, bar $\geq$ 10 %, n = 25; pass on the point), a judge's accuracy (94.2 %, CI 89.6–98.9 %, bar $\geq$ 95 %, n = 87; fail) and a judge threshold's false-intervention rate (2.4 %, CI 0.0–7.1 %, bar $\leq$ 5 %, n = 42; pass, but the threshold was fitted to meet that rate, so its interval is descriptive). The register keeps the point verdicts and reports the three as fragile. **Reproducibility of this study.** Every published number was recomputed from its published rows: 29 / 29 write-ups pass. From a fresh clone with no credentials, 6 gates rerun cold and reproduce (5 exactly). 12 are blocked by one unpublished software environment; 11 need a larger GPU, a private checkpoint, the arm or an unstated step. The register's citations and paper-side fields were audited against the papers' PDFs and the fixes are merged; of the 38 arXiv ids the published register names, the missing ones in our papers mirror belong to scheduled tests, and one tested paper, which has no arXiv id, was not checked against its PDF.

3 Results

Table 1 lists the register. Of the 22 papers, 4 reproduce, 13 fail, 3 are not proven, 1 was not tested and 1 was withdrawn. The register holds 45 claim rows (42 tested): 17 reproduce, 20 fail, 3 not proven, 4 not tested, 1 withdrawn. 8 of the 45 rows are our own baseline comparisons rather than paper claims (5 of them reproduce, e.g. "the commanded joints beat the world model"); on the 37 paper claims alone, 12 reproduce and 18 fail.

4 Three failure classes

We propose three classes for the 20 failed claim rows. **A, edge latency:** the method may work, but on the edge GPU it misses the real-time budget the claim needs; an adaptive chunking method takes 4,751 ms per decision against a budget of 298 ms. **B, zero-shot judges:** a model used without training on this rig to judge success, failure or progress is no better than a trivial signal; a learned reward model detects failures at AUROC 0.818 where episode length alone reaches 0.938. **C, latent planners beaten by joints:** a learned latent used to plan, rank or track progress does worse than the commanded joints or a copy-last / action-only baseline; a latent world model ranks the executed chunk first in 0.60 of anchors, against a bar of 0.8 and 0.68–0.90 for an action-only baseline. The three examples are illustrative, nothing more. Per-row labels wait: under the protocol committed before any reading, they count only once a blind second reader, labelling the same 20 rows on the same form, agrees with the first pass at Cohen's $\kappa \geq 0.6$. The protocol and the form are in the repository. The second reading has not happened yet.

5 Limitations

One rig, one arm, one operator. A failure here is a rig finding, not a refutation of the paper's own setting. The verdicts were labelled by one person, at rest, not always blind to the arm. Of the 89 gates, 6 are same-commit and 23 have no bar: for 29, the order of bar and result cannot be proven from this repository. The commit-order audit is mechanical, but its 21 upgrades rest on matching a bar text, and the second-reader check of the classes — the only independent check in the study — has not been run. Under the bootstrap, 3 verdicts are fragile. Only 6 gates were rerun cold. And the papers themselves were chosen for relevance to one stack, not sampled, so the reproduction rate describes this register, not the field.

6 Related work

A standardized real-world benchmark on the same arm reports a failure and recovery taxonomy for VLA policies [13]; a fleet of SO-101 cells evaluates policies at scale [14]. The vla-eval tool decouples VLA inference from 14 simulation benchmarks through a WebSocket protocol and Docker isolation [8], and a benchmark of demonstration quality studies one method family empirically [22]. These evaluate policies; we re-test the claims behind methods (judges, world models, latency) against bars fixed before the run, and audit our own pre-registration. The closest statement of our thesis [27] scores VLA and world-action models on five dimensions, among them reproduction evidence and deployment affordability; its reproduction-evidence score is five audit milestones (configuration mapping, task records, task coverage, training records, seeds) and, in its words, not a probability that a result is correct. We give one pass/fail per claim on a €1,350 rig, with the bar committed before the run and the pre-registration itself audited.

7 Conclusion

On a €1,350 rig, 12 of 37 tested paper claims reproduce under bars fixed for this hardware before the run. The scarce resource was not compute — $45.10 covered all of it — but order: bars before results. Auditing our own history found 29 of 89 gates whose order we cannot prove, and both numbers belong in the same paper. The register stays open. The next papers are already scheduled in the register file, and every number here can be rebuilt from the repository

paper	their rig	claim tested	bar	our result	verdict
LeWorldModel [9]	sim	A small JEPA world model learns the arm's dynamics well enough to plan with it.	Dynamics: beat copy-last, every corruption ≥ 1.3 at $K = 30$. Planning: rank the executed chunk first among 8 in ≥ 0.8 of anchors.	dynamics 0.31 vs 0.89 copy-last (reproduces); planning top-1 0.60 vs 0.68–0.90 for an action-only baseline (fail)	fail
DINO-WM [1]	sim	A predictor on frozen DINOv2 patches plans better than one on a learned latent.	G1b top-1 ≥ 0.8 at some target distance.	arm-pose R^2 0.92 (vs 0.57 for W1); G1b top-1 0.55–0.61; no-model baseline 0.68–0.90	fail
RoboJEPA [20]	>$20k	Pretrained RoboJEPA predicts our latents, uses the actions, and runs on the robot's computer.	9-step L1 ≥ 15 % below copy-last (300M) and ≥ 5 % below shuffled actions; 2-view encoder ≤ 200 ms on the Orin.	11.8 % below copy-last (300M; 1B 15.8 % on 30 clips); 9–10 % below shuffled; encoder 526 ms on the Orin NX	fail
H-JEPA [19]	sim	A slow second level makes latent distance track task progress.	Progress monotonicity Spearman $\rho \leq -0.80$; false stalls ≤ 10 %.	level-2 ρ -0.73; false stalls 52 %	fail
D-JEPA [18]	<$5k	A decision-aligned ranking operator on a frozen world model picks better action chunks.	G1b regret ≤ 0.10 and $\leq$ the action-only baseline + 0.05.	regret 0.263 (W1 alone 0.263; action-only -0.007)	not tested
ContactGuard [16]	$5–20k	An event-anchored probe on imagined latents predicts grasp failure better than the current latent.	AUROC ≥ 0.85 and ≥ 0.05 above the current latent.	0.70 imagined vs 0.82 current	fail
FAIL-Detect and SAFE [2] [5]	>$20k	A conformal band on a learned failure score keeps false alarms at α while catching failures.	Successes flagged $\leq 1/20$; failures flagged ≥ 50 %.	22.7 % of successes flagged at $\alpha = 5$ %; 46 % of failures caught	fail
ReAP [21]	$5–20k	A risk head on an action-conditioned latent warns before failures.	$\geq 5/8$ failures caught at $\leq 1/20$ false interventions; action-conditioned iff true $\geq$ shuffled + 0.05.	v1 AUROC 0.93 came from time and length; v2 0.789 vs 0.708 shuffled, 3/8 at 5/42	fail
VLA-Corrector [17]	sim	A verifier on observable history flags a failing rollout early enough to correct it.	$\geq 5/8$ failures at $\leq 1/20$ false interventions.	the paper's verifier family on our rollouts (a GRU on pooled RGB statistics and actions): 2/8 caught at 3/42 false interventions, AUROC 0.59; cosine drift of the W1 latent 0/8, AUROC 0.30	fail
Robometer [7]	>$20k	A 4B reward model judges SO-101 rollouts zero-shot.	Pooled AUROC ≥ 0.85 and within-policy ≥ 0.8 on ≥ 5 of 7 policies.	pooled AUROC 0.818 on 2099 ArmnetBench rollouts; episode length alone 0.938	fail
SmolVLA [3]	<$5k	A 450M VLA succeeds on an SO-101 task and can run on low-cost hardware.	10-trial operator-judged gate (no numeric pass mark registered); chunk inference within the async budget (667 ms).	9/10 on bowls; 974 ms in PyTorch, 238 ms with vla.cpp on the Orin NX	reproduces

Table 1: The register: one row per paper, generated from the register file (claim, bar and result as recorded, including our internal model and gate names); each re-tested paper is cited beside its name.

paper	their rig	claim tested	bar	our result	verdict
TurboVLA [15]	sim	Our claim that the architecture made the policy ignore its instruction.	Task swap $\geq 5°$ and per-layout $\geq$ 75 % $\rightarrow$ our port was the bug; $< 1°$ with the control $\geq 0.095 \rightarrow$ the data.	upstream's own trainer on our data: task swap 0.020°, per-layout 38.9 %; control, their released LIBERO checkpoint run here: 0.105 ($\approx 5.5°$ of pan)	withdrawn
VLA-JEPA [6]	sim	VLA-JEPA trains cheaply on SO-101 data and runs on the robot.	10-trial operator-judged gate (no numeric pass mark registered); for reranking (H0): oracle / mean error $\leq$ 0.7 and N = 8 $\leq 1.5\times$ the latency.	9/10 for $9.7; 355 ms per chunk on the Orin NX; reranking no-go (0.76, 1.84×)	reproduces
REKI [23]	sim	Policies without memory oscillate between poses at a measurable rate.	$\geq$ 10 % of failed episodes oscillate (their criterion).	20.3 % of all episodes, 24.0 % of failures; REKI trained on my data: 19/20 + 19/20 on the arm, oscillates in 1/40	reproduces
Real-Time Chunking [4]	unstated	Conditioning a new chunk on the executing one removes the jump at chunk boundaries.	Best gradient-free method cuts the median jump $\geq$ 50 % against naive async, drift $\leq$ +10 %, within 667 ms.	prefix overwrite cuts the jump 71–73 % against naive async at +0.4 ms; tuned LeRobot identity mode is close	not proven
HWM (hierarchical latent world models) [10]	>$20k	A high-level world model over learned macro-actions beats the low-level model beyond about 1 s (their Fig. 6 crossover).	Crossover in [1.0, 2.0] s; at 2 s high-level error $\leq$ 0.95 × low-level; learned macro-actions $\geq$ net joint displacement on cosine.	no crossover: W1 $0.382 \pm 0.005 \rightarrow 0.563 \pm 0.007$, high level $0.769 \pm 0.009 \rightarrow 0.759 \pm 0.009$ (0.73–2 s); cosine 0.608 ± 0.007 vs 0.551 ± 0.006	not proven
LeJEPA identifiability [12]	sim	A SIGReg JEPA trained on goal-directed data recovers the true latent state only partially and unevenly.	W1's mean R^2 from latent to joints < 0.80; anisotropy (max − min per-joint R^2) ≥ 0.20; the higher-R^2 model has the higher G1b top-1.	W1 mean R^2 0.565, anisotropy 0.639 (shoulder lift 0.91, wrist flex 0.27); frozen DINOv2-S 0.867 / 0.301	reproduces
SURVEYOR	sim	A threshold on the imagined-vs-reached latent distance accepts good plans and rejects bad ones ($\tau = 0.20$).	τ below the 10th percentile of non-equivalent pairs and ≤ 0.40; as a monitor $\geq$ 5/8 failures at $\leq$ 2/42; $\geq$ 0.05 AUROC over a shuffled drafter.	$\tau = 1.249$ on W1 latents of 69 real rollouts; as a monitor 3/8 at 6/42 (S = 30), AUROC 0.756 vs shuffled 0.560	fail
Adaptive Action Chunking (AAC) [11]	>$20k	Chunk length chosen from the entropy of N sampled chunks follows the task's phases (long in transport, short at the grasp), lowers the boundary jump, and costs little with batched sampling.	median h* in transport $\geq$ grasp window + 2; jump with AAC + overwrite $\leq$ fixed + overwrite at d = 8; an N = 20 decision $\leq$ 298 ms on the Orin.	h* 2 in transport and 2 at the grasp (86 % of decisions at the floor); jump 0.973 vs 0.940; N = 20 decision 4,751 ms (sequential) on the Orin	fail
PACT-VLA [24]	$5–20k	Picking among N sampled chunks by their flow residual under the pre-commit observation fits what happens next better than a random or stale pick, with no added control-step delay.	PACT beats a random pick and the stale-observation residual at d = 8 (CIs < 0); its delay on the Orin $\leq$ the naive delay; at that delay it still beats a single sample.	offline $-0.27°$ vs random and vs the stale residual (both CIs < 0); on the Orin 1,200 ms per cycle, 37 control ticks vs 8	fail
Horizon × depth (Euler steps vs execution horizon) [25]	>$20k	With long open-loop commitment, cutting a flow-matching VLA's Euler steps from 10 to 2 contracts the action cloud around an unchanged centroid, saves most of the inference time and avoids edge-GPU thermal throttling.	median R(2)/R(10) $\leq$ 0.70 with the centroid within 0.05 relative and 3°; N = 2 latency $\leq 0.85 \times$ N = 10 on the Orin, N = 10 within 5 % of 238 ms.	R(2)/R(10) 0.78 (CI 0.73–0.81), centroid 0.018 and 2.35°; 196.1 vs 237.5 ms per chunk (0.83, vision 151 ms); no throttling at N = 10 on the Orin, at MAXN_SUPER or 25 W	not proven
REMORA [26]	>$20k	Reading a progress distribution out at the supported peak nearest the previous estimate steadies the estimate and lets it fall during real setbacks, with no worse detection of the stages.	Against per-frame argmax: premature not higher and missed $\leq$ +2 points (P1); flips halved, CI upper $\leq$ 0.7 (P2); a decrease inside $\geq$ 50 % of the reversal windows with $\leq$ 20 % false regressions (P3).	premature 0.22 vs 0.25, missed equal (P1 pass); flips 0.70 × argmax, CI 0.48–0.94 (P2 fail); setback recall 2/7, false 0.52 (P3 fail); MAE 0.184 vs median 0.218	fail

Table 1, continued.

(github.com/ravediamond/paper-register-reproducibility).

References

[1] DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning, 2024. arXiv:2411.04983.

[2] Can We Detect Failures Without Failure Data? Uncertainty-Aware Runtime Failure Detection for Imitation Learning Policies, 2025. arXiv:2503.08558.

[3] SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics, 2025. arXiv:2506.01844.

[4] Real-Time Execution of Action Chunking Flow Policies, 2025. arXiv:2506.07339.

[5] SAFE: Multitask Failure Detection for Vision-Language-Action Models, 2025. arXiv:2506.09937.

[6] VLA-JEPA: Enhancing Vision-Language-Action Model with Latent World Model, 2026. arXiv:2602.10098.

[7] Robometer: Scaling General-Purpose Robotic Reward Models via Trajectory Comparisons, 2026. arXiv:2603.02115.

[8] vla-eval: A Unified Evaluation Harness for Vision-Language-Action Models, 2026. arXiv:2603.13966.

[9] LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels, 2026. arXiv:2603.19312.

[10] Hierarchical Planning with Latent World Models, 2026. arXiv:2604.03208.

[11] Adaptive Action Chunking at Inference-time for Vision-Language-Action Models, 2026. arXiv:2604.04161.

[12] When Does LeJEPA Learn a World Model?, 2026. arXiv:2605.26379.

[13] Benchmarking Vision-Language-Action Models on SO-101: Failure and Recovery Analysis, 2026. arXiv:2606.08881.

[14] ArmnetBench v0.1: Parallel Real-World Evaluation of Manipulation Policies on a Low-Cost Arm Farm, 2026. arXiv:2607.24481.

[15] TurboVLA: Real-Time Vision-Language-Action Model at 32 Hz on an RTX 4090 with <1 GB VRAM, 2026. arXiv:2607.27205.

[16] ContactGuard: Pre-Contact Execution Monitoring with Action-Conditioned Latent World Models, 2026. arXiv:2608.13438.

[17] VLA-Corrector: Stage-Aware Observable State Understanding for Prompt-Based Closed-Loop Recovery of Vision-Language-Action Policies, 2026. arXiv:2609.06508.

[18] D-JEPA: A Decision-Aligned Latent World Model, 2026. arXiv:2609.24749.

[19] H-JEPA: End-to-End Learning of Hierarchical World Models for Visual Planning, 2026. arXiv:2610.06805.

[20] RoboJEPA: Scaling Robotic Latent World Models, 2026. arXiv:2610.10515.

[21] ReAP: From Visual Failure Recognition to Action-Conditioned Risk Prediction for Runtime Failure Monitoring, 2026. arXiv:2610.16309.

[22] What Matters in Quality-Aware VLA Pretraining: A Benchmark and Empirical Study, 2026. arXiv:2610.18869.

[23] REKI: Registering Knowledge in VLAs, 2026. arXiv:2610.22178.

[24] PACT-VLA: Speculative Action-Chunk Verification with Pre-Commit Observations, 2026. arXiv:2610.28019.

[25] Coupling Execution Horizon and Sampling Depth in Flow-Matching VLA Policies, 2026. arXiv:2610.32102.

[26] Tracking Task Progress Through Errors and Recovery:A Benchmark and Recurrent Distributional Model, 2026. arXiv:2610.39125.

[27] Xin Ma et al. RoboSeed: Multidimensional Evaluation of Vision-Language-Action and World-Action Models, 2026. arXiv:2610.40026.